جامعة بيرزيت

BIRZEIT UNIVERSITY

Computer Vision - COMP9320

Assignments #1

Submission deadline: 13.11.2026

Objective

The objective of this assignment is to implement and analyze fundamental digital image processing techniques using Python and OpenCV. You will explore image smoothing, noise reduction, edge detection, and multiresolution image blending.

Submission Requirements

- Write Python code to complete all the tasks.
- Use a **Jupyter Notebook (.ipynb)** to implement and document your solutions.
- Include the input images, processed results, and appropriate visualizations.
- Add brief explanations of the methods used and observations about the results.
- Submit the completed Jupyter Notebook.

Task 1: Image Loading and Grayscale Conversion

1. Use OpenCV to read the image uni.jpeg.
2. Display the original image and report its:
    - Spatial dimensions.
    - Number of color channels.
    - Data type and numerical intensity range.
    - Memory requirements.
3. Convert the original image to grayscale.
4. Display the grayscale image.

Task 2: Edge Detection Using Canny

1. Use the grayscale image obtained in Task 1.
2. Apply the Canny edge detection algorithm to extract the edges.
3. Display the grayscale image and the resulting edge image side by side.
4. Explain the five fundamental stages of the Canny algorithm:
    - Gaussian smoothing.
    - Gradient computation.
    - Gradient magnitude and orientation estimation.
    - Non-maximum suppression.
    - Hysteresis thresholding.
5. Report suitable lower and upper hysteresis thresholds and justify their selection.

6. Investigate the effect of changing the lower and upper thresholds on edge connectivity, localization, and false-positive detections.

Task 3: Gaussian Filtering and Comparison

1. Apply a Gaussian filter to the grayscale image from Task 1 using a Gaussian kernel with a standard deviation of $\sigma = 2\sqrt{2}$.
2. Apply a Gaussian filter twice to the original grayscale image, using $\sigma = 2$ for each filtering operation.
3. Display both resulting images.
4. Compute the absolute pixel-wise difference between the two filtered images.
5. Display the difference image and briefly explain the observed results.
6. Investigate why the two implementations may produce non-zero differences in practice. Consider:
    - Finite kernel truncation.
    - Discrete sampling.
    - Kernel normalization.
    - Boundary handling.
    - Numerical precision.

Task 4: Salt-and-Pepper Noise Removal

1. Read uni.jpeg and convert it to grayscale.
2. Add **30% salt-and-pepper noise**, meaning that 30% of the pixels should be randomly selected and replaced with either black or white values.
3. Display the noisy image.
4. Apply the following filters to the noisy image:
   (a) Gaussian filter
   (b) Median filter
   (c) Bilateral filter
5. Select suitable parameters for each filter to obtain the best visual results.
6. Display the three filtered images side by side.
7. Compare the results and discuss which filter removes salt-and-pepper noise most effectively while preserving image details and edges.
8. Repeat the experiments using at least three different noise densities, including 10%, 30%, and 50%.
9. Analyze why Gaussian filtering is generally suboptimal for impulse noise removal.

Task 5: Image Blending Using Laplacian Pyramids

Blend the images apple.jpg and orange.jpg using **Laplacian pyramid blending** to produce a seamless composite image similar to Figure 1.

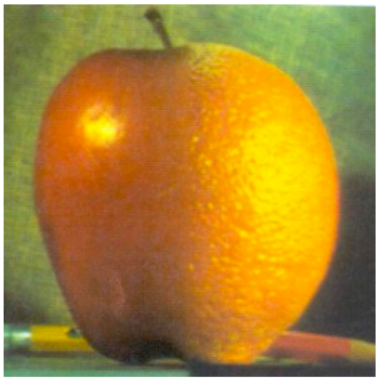


Figure 1: Blended image.

Your implementation should include the following steps:

1. Read apple.jpg and orange.jpg.
2. Ensure that both images have compatible dimensions.
3. Construct Gaussian pyramids for both images.
4. Construct Laplacian pyramids from the Gaussian pyramids.
5. Create a blending mask and construct its Gaussian pyramid.
6. Blend the corresponding levels of the Laplacian pyramids using the mask pyramid.
7. Reconstruct the final blended image from the combined Laplacian pyramid.
8. Display the original images and the final blended result.

Reference: P. Burt and E. Adelson, "A Multiresolution Spline With Application to Image Mosaics," *ACM Transactions on Graphics*, 1983.

General Guidelines

- Use Python and OpenCV (cv2) for image processing.
- You may use NumPy for numerical operations and Matplotlib for visualization.
- Label all displayed images clearly.
- Ensure that the notebook cells are organized, executable, and accompanied by brief explanations.
- Include a short conclusion summarizing the main findings from the filtering, edge detection, noise removal, and image blending experiments.

Rubrics

Component	Marks
Task 1: Image loading and grayscale conversion	10
Task 2: Canny edge detection and interpretation	10
Task 3: Gaussian filtering and difference analysis	20
Task 4: Noise generation, filtering, and comparison	25
Task 5: Laplacian pyramid blending	25
Notebook organization, explanations, and conclusion	10
Total	100